

Task 1 - SAST Tools

1. Python Application Findings

A. High Severity: Flask Debug Mode Enabled (CWE-94)

Location: `./bad/vulpy.py:55:0`

Code Snippet:

```
app.run(debug=True, host='127.0.1.1', port=5000, extra_files='csp.txt')
```

Issue:

Debug mode exposes the Werkzeug debugger, allowing attackers to execute arbitrary code if accessed. Critical for publicly accessible apps.

Mitigation:

Disable debug mode in production:

```
app.run(debug=False, host='127.0.1.1', port=5000)
```

B. Medium Severity: SQL Injection (CWE-89)

Location: `./bad/libuser.py:12:21`

Code Snippet:

```
user = c.execute("SELECT * FROM users WHERE username = '{}' and password = '{}'"
                  .format(username, password))
```

Issue:

Direct user input embedding enables SQL injection (e.g., `' OR 1=1 --`).

Mitigation:

Use parameterized queries:

```
user = c.execute("SELECT * FROM users WHERE username = ? AND password = ?",
                  (username, password))
```

C. Low Severity: Hardcoded Password (CWE-259)

Location: `./bad/vulpy.py:16:11`

Code Snippet:

```
app.config['SECRET_KEY'] = 'aaaaaaa'
```

Issue:

Hardcoded secrets risk exposure if code is leaked.

Mitigation:

Use environment variables:

```
import os
app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY')
```

Python App Summary:

Severity	Issue	CWE	Mitigation
High	Debug Mode Enabled	94	Disable <code>debug=True</code>
Medium	SQL Injection	89	Use parameterized queries
Low	Hardcoded Secret	259	Use environment variables

2. C Program Findings

A. Level 1 Vulnerability: Buffer Over-Read via `strlen` (CWE-126)

Location: `./libAFL/damn_vulnerable_c_program_shmem/imgRead.c:35`

Code Snippet:

```
bufsize = strlen(shmem_buf);
```

Issue:

Potential buffer over-read if `shmem_buf` lacks null termination.

Mitigation:

Validate null termination or use `memchr` to check for `\0`.

B. Level 2 Vulnerability: Unsafe `memcpy` (CWE-120)

Location: `./dvcp.c:58`

Code Snippet:

```
memcpy(buff1, img.data, sizeof(img.data));
```

Issue:

Buffer overflow risk if `img.data` exceeds `buff1` size.

Mitigation:

Add size checks or use `memcpy_s`:

```
if (sizeof(img.data) <= sizeof(buff1)) {
    memcpy(buff1, img.data, sizeof(img.data));
}
```

There is no level 3 Vulnerability found

C. False-Positive Example

Flagged Code:

```
fp = fopen(filename, "r");
```

Location: `./dvcp.c:33`

Why False-Positive:

Hardcoded paths (e.g., `fopen("/etc/static_config.conf", "r")`) pose minimal risk.

C Program Summary:

Level	Vulnerability	CWE	Mitigation
1	Buffer Over-Read	126	Validate null termination
2	Unsafe <code>memcpy</code>	120	Check buffer sizes

3. JavaScript Application Findings

A. ERROR Severity: XSS in EJS Template (CWE-79)

Location: `views/app/products.ejs`

Code Snippet:

```
<%- output.products[i].description %>
```

Issue:

Unsanitized user input enables script injection.

Mitigation:

Use `<%= ... %>` for auto-escaping or sanitize with `DOMPurify`.

B. WARNING Severity: Cleartext Database Connection (CWE-319)

Location: `config/db.js`

Code Snippet:

```
dialect: 'mysql'
```

Issue:

Unencrypted database traffic risks MITM attacks.

Mitigation:

Enable TLS:

```
dialectOptions: { ssl: { require: true, rejectUnauthorized: false } }
```

C. INFO Severity: Default Session Cookie (CWE-522)

Location: `server.js`

Code Snippet:

```
app.use(session({ secret: 'keyboard cat', cookie: { secure: false } })))
```

Issue:

Predictable cookie name and insecure settings.

Mitigation:

Customize cookie name and enforce HTTPS:

```
name: 'customSessionId', cookie: { secure: true }
```

JavaScript App Summary:

Severity	Vulnerability	CWE	Mitigation
ERROR	XSS in EJS Template	79	Use <%= %> or sanitize HTML
WARNING	Cleartext Database Connection	319	Enable SSL/TLS
INFO	Default Session Cookie	522	Customize name, enable secure

2

Task 2 - Web Security Mini Labs

Student Alias: [YehiaSobeh]

1. Cross-Site Scripting (XSS)

Steps Exploited

1. Deployed the vulnerable XSS application using Docker:

```
docker run -p 127.0.0.1:5000:5000 sh3b0/vuln:xss
```

2. Injected the following script into a comment or search field:

```
<script>alert('YehiaSobehSTUDENT123')</script>
```

3. Successfully triggered an alert displaying the student ID.

Screenshot:

Why It's Dangerous

- Attackers can execute malicious scripts in users' browsers.
- This can lead to session hijacking, phishing attacks, and cookie theft.

Protection Measures

- Sanitize user input by escaping special characters such as <, >, and &.

- Implement Content Security Policy (CSP) headers to restrict script execution.
-

2. Path Traversal

Steps Exploited

1. Ran the vulnerable Path Traversal application:

```
docker run -p 127.0.0.1:5000:5000 sh3b0/vuln:path-traversal
```

2. Accessed a sensitive file by visiting:

```
http://localhost:5000
```

3. modify the path inside the html from `text/intro.txt` into `../../../../etc/passwd`
4. Successfully retrieved the `/etc/passwd` file, proving the vulnerability.

Screenshot:

Why It's Dangerous

- Attackers can access sensitive files, including configuration files and credentials.

Protection Measures

- Validate and sanitize user-supplied file paths.
 - Implement a whitelist of allowed files.
-

3. SQL Injection

Steps Exploited

1. Launched the SQL Injection test application:

```
docker run -p 127.0.0.1:5000:5000 sh3b0/vuln:sqli
```

2. Injected a malicious SQL payload:

```
UNION SELECT * FROM users
```

3. Extracted sensitive database information, including a fake admin account.

Screenshot:**Why It's Dangerous**

- Allows unauthorized database access, data leaks, and even data deletion.

Protection Measures

- Always use parameterized queries or ORM libraries.
 - Escape special characters in user inputs.
-

4. File Upload Vulnerability

Steps Exploited

1. Started the vulnerable file upload application:

```
docker run -p 127.0.0.1:5000:5000 sh3b0/vuln:file-upload
```

2. Uploaded a html file having in the page (hello yehia):

```
../templates/index.html
```

3. Fetch the home page:

```
http://localhost:5000
```

Screenshot:**Why It's Dangerous**

- Enables remote code execution (RCE), allowing attackers full control over the server.

Protection Measures

- Restrict file uploads to specific extensions (e.g., `.jpg`, `.png`).
 - Store uploaded files outside the webroot to prevent direct execution.
-

5. Command Injection

Steps Exploited

1. Launched the command injection test application:

```
docker run -p 127.0.0.1:5000:5000 sh3b0/vuln:command-injection
```

2. Injected a malicious command into the ping utility:

```
50 %; rm -rf static/img/bones.png
```

3. The output confirmed successful command execution.

Screenshot:

Why It's Dangerous

- Attackers can execute arbitrary system commands, potentially taking over the server.

Protection Measures

- Avoid passing user input directly to system commands.
- Implement strict input validation and whitelisting.

General Best Practices for Web Security

- **Input Validation:** Ensure all user input is properly sanitized and validated.
- **Principle of Least Privilege:** Restrict file and database permissions to minimize damage from exploits.
- **Security Headers:** Implement security-focused HTTP headers like CSP and **X-Content-Type-Options**.
- **Secure Coding Practices:** Use parameterized queries, avoid functions like `eval()`, and never execute system commands with unsanitized input.
- **Regular Security Audits:** Conduct penetration testing and code reviews periodically to detect and mitigate vulnerabilities before attackers can exploit them.